NIVEL 1

Exercici 1: Consulta sobre Taula no Optimitzada (Diagnòstic).

Archivo Editar Ejecutar Herramientas

```
SELECT *
FROM `sprint3-analytics-gene.sprint3_silver.companies_clean` AS co
JOIN `sprint3-analytics-gene.sprint3_silver.transactions_clean` AS t ON co.company_id = t.business_id
WHERE EXTRACT(DATE FROM t.timestamp) = '2022-03-12'
AND co.country='Germany';
```

Esta consulta procesará 11.86 MB cuando se ejecute.

Exercici 2: Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster).

Pas 1: Generació de Dades Recents (Mocking Data)

Creamos la tabla intermedia transactions_recent desde transactions_clean

Consulta sin título | Ejecutar | Guardar | Programa | Compartir

Archivo Editar Ejecutar Herramientas

```
CREATE OR REPLACE TABLE `sprint3_silver.transactions_recent` AS
SELECT * EXCEPT(timestamp),
TIMESTAMP_SUB(CURRENT_TIMESTAMP(),INTERVAL CAST(RAND() * 50 AS INT64)DAY) AS timestamp
FROM `sprint3-analytics-gene.sprint3_silver.transactions_clean`;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda Ubicación de procesamiento: EU

Recientes | Resultados de la consulta

6:55 p.m.

CREATE OR REPLACE T...

1:02 p.m.

Información del trabajo | Resultados | Detalles de la ejecución | Gráfic

Esta declaración creó una nueva tabla con el nombre transactions_recent.

Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering)

Consulta sin título | Ejecutar | Guardar | Programa | Compartir

Archivo Editar Ejecutar Herramientas

```
CREATE OR REPLACE TABLE `sprint3_gold.fact_transactions_optimized`
PARTITION BY DATE(timestamp)
CLUSTER BY business_id AS
SELECT *
FROM `sprint3_silver.transactions_recent`;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Recientes | Resultados de la consulta

7:09 p.m.

CREATE OR REPLACE T...

Información del trabajo | Resultados | Detalles de la ejecución | Gráfico de

Esta declaración creó una nueva tabla con el nombre fact_transactions_optimized.

Confirmamos con vista previa que este correcto:

fact_transactions_optimized | Consulta | Chat | Abrir en | Compartir | Copiar | Instantánea | Borrar | Exportar o sincronizar

Esta es una tabla particionada. Learn more

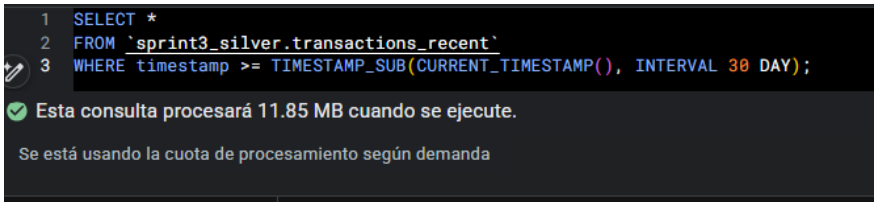
Esquema | Detalles | Vista previa | Estadísticas | Linaje | Perfil de datos | Calidad de los datos

	transaction_id	business_id	card_id	amount	declined	product_ids	user_id	lat	longitude	timestamp
1	9C07B3D5-865F-4665-9792-08...	b-2222	CcS-8744	300.76	0	65 78	4163	40.9564581...	-3.65265108...	2026-09-30 16:55:19.652080 UTC
2	67F02E9B-BEA2-4C48-B95A-63...	b-2222	CcS-5018	339.18	0	60 56 71	437	51.7920419...	5.37985337...	2026-09-30 16:55:19.652080 UTC
3	9C09F20F-939C-413F-9B46-95...	b-2222	CcS-5066	238.04	0	32 91	485	51.6950496...	19.0500558...	2026-09-30 16:55:19.652080 UTC
4	9F910731-8073-4D13-BDCE-EE...	b-2222	CcS-8066	260.47	0	63 4	3485	55.5603784...	-3.51237953...	2026-09-30 16:55:19.652080 UTC
5	634817BD-580C-4079-9D22-E5...	b-2222	CcS-5034	65.25	0	97	453	51.5583917...	10.3960969...	2026-09-30 16:55:19.652080 UTC

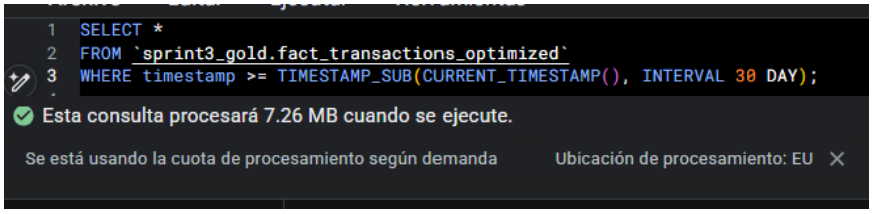
Confirmamos que este particionada y agrupada en clústeres:

Exercici 3: La Prova del Cotó (Benchmark).

Pas 1 (Taula no optimitzada):



Pas 2 (Taula optimitzada):

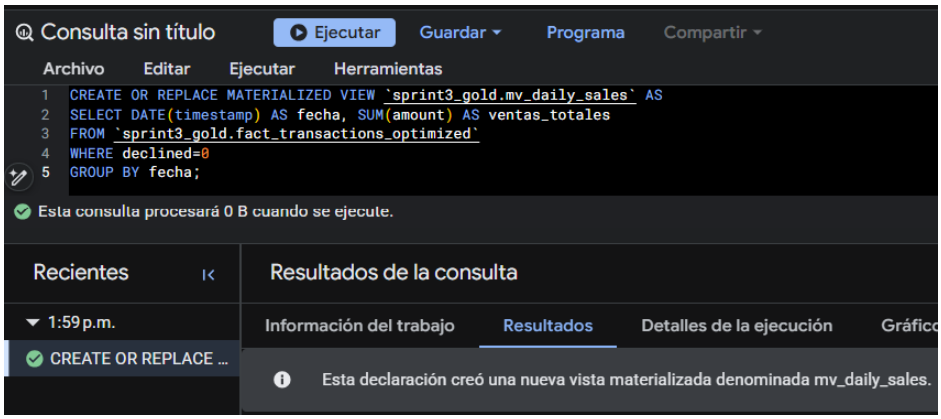


La consulta sobre la tabla no optimizada sprint3_silver.transactions_recent supone un procesamiento estimado de 11,85 MB. Al realizar la misma consulta sobre la tabla optimizada sprint3_gold.fact_transactions_optimized, particionada por fecha, los bytes procesados se reducen a 7,26 MB.

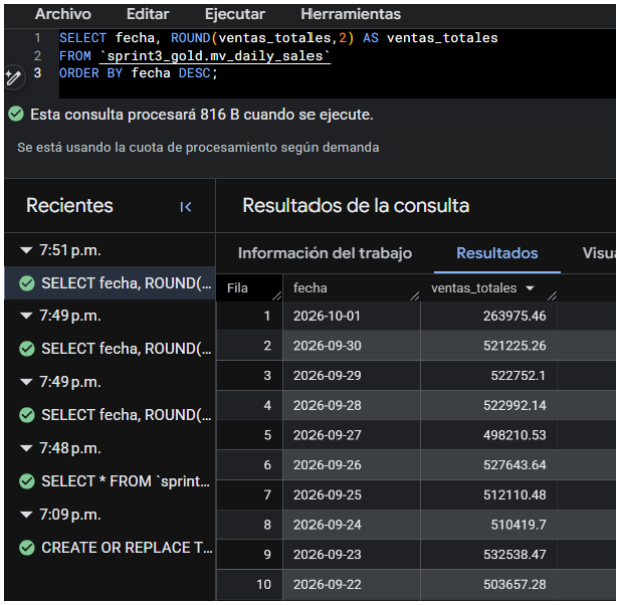
Esto supone un ahorro de 4,59 MB, equivalente a un 38,73 % menos de datos procesados. La reducción se debe principalmente al particionamiento por DATE(timestamp), que permite a BigQuery descartar las particiones que quedan fuera del periodo consultado.

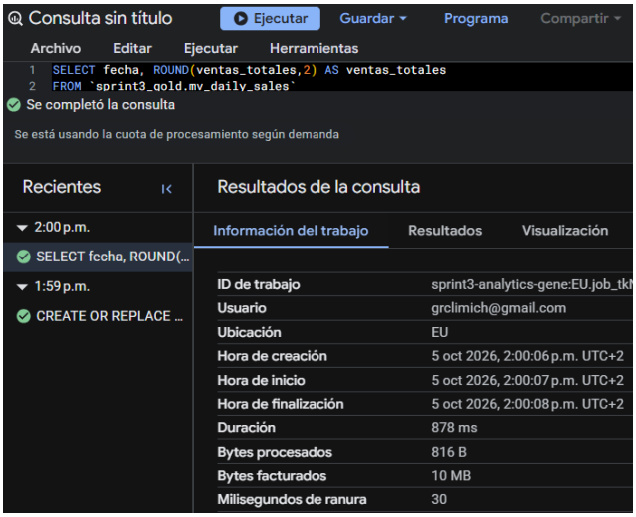
Exercici 4: Smart Caching (Vistes Materialitzades).

Creamos nuestra vista materializada:



Consultamos los datos de la vista y verificamos los bytes procesados:

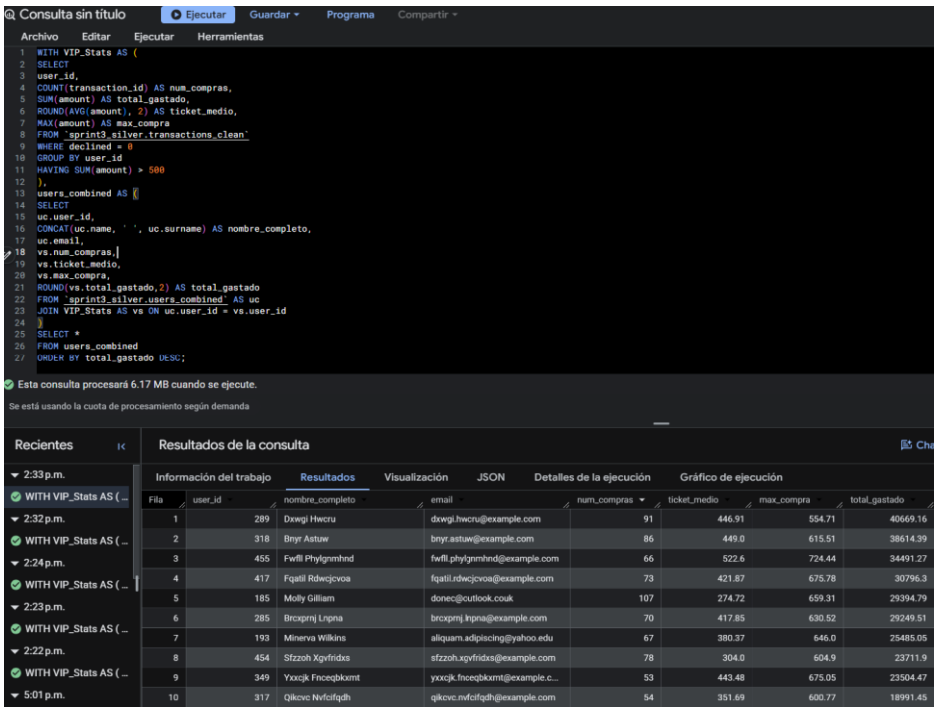




Nivell 2: SQL Analític Avançat

Exercici 1: Perfilat de Clients VIP (Mètriques Agregades CTE)

Identificamos a los usuarios que han gastado más de 500 € en transacciones válidas (declined >= 0). Para cada usuario calculamos el número de transacciones, ventas totales, ticket medio y compra máxima. Después unimos esta información con los datos personales del usuario y mostramos los clientes ordenados de mayor a menor gasto.



```
WITH VIP_Stats AS (
SELECT
user_id,
COUNT(transaction_id) AS num_compras,
SUM(amount) AS total_gastado,
ROUND(AVG(amount), 2) AS ticket_medio,
MAX(amount) AS max_compra
FROM `sprint3_silver.transactions_clean`
WHERE declined = 0
GROUP BY user_id
HAVING SUM(amount) > 500
),
users_combined AS (
SELECT
uc.user_id,
CONCAT(uc.name, ' ', uc.surname) AS nombre_completo,
uc.email,
vs.num_compras,
vs.ticket_medio,
vs.max_compra,
ROUND(vs.total_gastado,2) AS total_gastado
FROM `sprint3_silver.users_combined` AS uc
JOIN VIP_Stats AS vs ON uc.user_id = vs.user_id
)
SELECT *
FROM users_combined
ORDER BY total_gastado DESC;
```

Esta consulta procesará 6.17 MB cuando se ejecute.

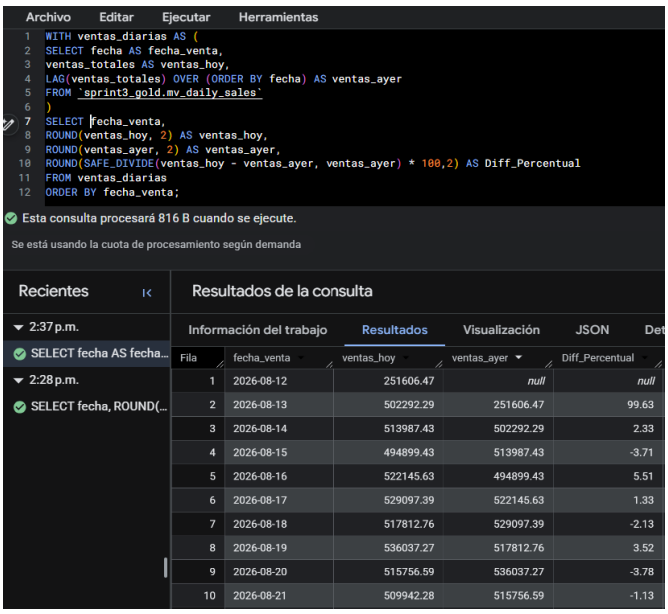
Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Fila	user_id	nombre_completo	email	num_compras	ticket_medio	max_compra	total_gastado
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.39
3	455	Fwfll Phylgnmhnd	fwfll.phylgnmhnd@example.com	66	522.6	724.44	34491.27
4	417	Fqatil Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	73	421.87	675.78	30796.3
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.79
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	78	304.0	604.9	23711.9
9	349	Yxxcjk Fnceqbkxmt	yxxcjk.fnceqbkxmt@example.c...	53	443.48	675.05	23504.47
10	317	Qikcvc Nvfcifqdh	qikcvc.nvfcifqdh@example.com	54	351.69	600.77	18991.45

Exercici 2: Anàlisi de Tendències (Window Functions/vistes)

Comparamos las ventas de cada día con las del día anterior. Utilizamos LAG() para obtener las ventas del día anterior y calculamos la diferencia porcentual entre ambos días. De esta forma podemos analizar cómo evolucionan las ventas diariamente.



```
WITH ventas_diarias AS (
SELECT fecha AS fecha_venta,
ventas_totales AS ventas_hoy,
LAG(ventas_totales) OVER (ORDER BY fecha) AS ventas_ayer
FROM `sprint3_gold.mv_daily_sales`
)
SELECT fecha_venta,
ROUND(ventas_hoy, 2) AS ventas_hoy,
ROUND(ventas_ayer, 2) AS ventas_ayer,
ROUND(SAFE_DIVIDE(ventas_hoy - ventas_ayer, ventas_ayer) * 100,2) AS Diff_Percentual
FROM ventas_diarias
ORDER BY fecha_venta;
```

Esta consulta procesará 816 B cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Fila	fecha_venta	ventas_hoy	ventas_ayer	Diff_Percentual
1	2026-08-12	251606.47	null	null
2	2026-08-13	502292.29	251606.47	99.63
3	2026-08-14	513987.43	502292.29	2.33
4	2026-08-15	494899.43	513987.43	-3.71
5	2026-08-16	522145.63	494899.43	5.51
6	2026-08-17	529097.39	522145.63	1.33
7	2026-08-18	517812.76	529097.39	-2.13
8	2026-08-19	536037.27	517812.76	3.52
9	2026-08-20	515756.59	536037.27	-3.78
10	2026-08-21	509942.28	515756.59	-1.13

Exercici 3: Totals Acumulats (Running Totals sobre Vistes)

Calculamos las ventas acumuladas desde el comienzo de cada año hasta cada fecha (YTD). Para ello utilizamos una función de ventana que acumula las ventas diarias, reiniciando el cálculo cuando comienza un nuevo año.

```
SELECT fecha AS fecha_venta,
ROUND(ventas_totales,2) as ventas_hoy,
ROUND(SUM(ventas_totales) OVER (PARTITION BY EXTRACT(YEAR FROM fecha) ORDER BY fecha ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW),2) AS ventas_acumuladas_YTD
FROM `sprint3_gold.mv_daily_sales`
ORDER BY fecha_venta;
```

Esta consulta procesará 816 B cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Recientes

- 2:47 p.m. SELECT fecha, ROUN...
- 10:47 p.m. SELECT fecha, ROUN...
- 10:44 p.m. SELECT fecha, ROUN...
- 7:30 p.m. SELECT user_id, EXT...
- 7:29 p.m. SELECT user_id, EXT...
- 7:29 p.m.

Resultados de la consulta

Información del trabajo | Resultados | Visualización | JSON | Detalles de la ejecución | Gráfico de ejecución

Fila	fecha	ventas_hoy	ventas_acumuladas_YTD
1	2026-08-12	251606.47	251606.47
2	2026-08-13	502292.29	753898.76
3	2026-08-14	513987.43	1267886.19
4	2026-08-15	494899.43	1762785.62
5	2026-08-16	522145.63	2284931.25
6	2026-08-17	529097.39	2814028.64
7	2026-08-18	517812.76	3331841.4
8	2026-08-19	536037.27	3867878.67
9	2026-08-20	515756.59	4383635.26
10	2026-08-21	509942.28	4893577.54

Exercici 4: Fidelització i Valor del Client (Filtratge Avançat)

Identificamos a los usuarios que han realizado al menos tres compras válidas. Primero ordenamos cronológicamente sus transacciones mediante ROW_NUMBER, conservamos únicamente las tres primeras y calculamos el ticket medio de esas tres compras. Finalmente, mediante QUALIFY, mostramos únicamente la tercera compra de cada usuario junto con su fecha, importe y media de gasto inicial.

Archivo Editar Ejecutar Herramientas

```
WITH segmentacion_cliente AS (
SELECT
user_id, EXTRACT(DATE FROM timestamp) AS fecha,
ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp) as orden_compras,
amount AS compras
FROM `sprint3_silver.transactions_clean`
WHERE declined = 0
),
primeras3_compras AS(
SELECT  user_id,fecha,orden_compras,compras
FROM segmentacion_cliente
WHERE orden_compras <=3 -- filtramos primero las primeras 3 compras por cliente
)
SELECT pr.user_id, CONCAT(uc.name, ' ' ,uc.surname) as nombre_completo, uc.email,
pr.fecha AS fecha_3ra_compra, pr.compras AS importe_3ra_compra,
ROUND(AVG(pr.compras) OVER (PARTITION BY pr.user_id),2) AS avg_primeras3_compras -- hacemos el promedio solo de las 3 primeras compras por cliente
FROM `primeras3_compras` AS pr
JOIN `sprint3_silver.users_combined` AS uc ON pr.user_id = uc.user_id
QUALIFY pr.orden_compras = 3; -- Solo clientes que hayan completado 3 compras
```

Esta consulta procesará 3.31 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Recientes

- 3:00 p.m. WITH segmentacion...
- 10:55 p.m. WITH segmentacion...
- 10:50 p.m. WITH segmentacion...
- 10:39 p.m. WITH segmentacion...
- 10:38 p.m. WITH segmentacion...
- 10:38 p.m.

Resultados de la consulta

Información del trabajo | Resultados | Visualización | JSON | Detalles de la ejecución | Gráfico de ejecución

Fila	user_id	nombre_completo	email	fecha_3ra_compra	importe_3ra_compra	avg_primeras3_compras
1	1	Zeus Gamble	interdum.enim@protonmail.edu	2015-12-20	161.33	239.13
2	2	Garrett Mcconnell	integer.vitae.nibh@protonmail.o...	2015-09-21	267.6	287.46
3	3	Ciaran Harrison	interdum.feugiat@aol.org	2016-01-22	59.76	166.44
4	4	Howard Stafford	ornare.egestas@icloud.edu	2016-09-28	171.13	225.76
5	5	Hayfa Pierce	et.malesuada.fames@hotmail.o...	2015-08-17	51.94	184.49
6	6	Joel Tyson	gravida.nunc.sed@yahoo.ca	2015-05-22	386.09	316.65
7	7	Rafael Jimenez	eget@outlook.ca	2015-07-18	251.42	282.43
8	8	Nissim Franks	egestas.aliquam.fringilla@goog...	2016-04-16	410.88	337.88
9	9	Mannix Mcclain	aliquam.nisl@outlook.com	2015-07-21	139.59	90.94
10	10	Robert Mccarthy	fermentum@protonmail.com	2015-11-08	470.05	365.8

Nivell 3: Analytics Engineering (Arrays & Automatització)

Exercici 1: Desanidament i Aplanament de Dades (Unnesting):

Creamos la tabla haciendo un cross join con unnest, El CROSS JOIN permite asociar cada elemento resultante del UNNEST con la fila original de la transacción.

Archivo Editar Ejecutar Herramientas

```sql
CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
SELECT
tc.transaction_id,
tc.business_id,
tc.card_id,
tc.timestamp,
tc.amount,
tc.declined,
tc.product_ids,
tc.user_id,
tc.lat,
tc.longitude,
pc.product_id AS product_sku,
pc.name AS product_name,
pc.price AS product_price
FROM `sprint3_silver.transactions_clean` AS tc
CROSS JOIN UNNEST(tc.product_ids) AS product_id
LEFT JOIN `sprint3_silver.products_clean` AS pc ON product_id = pc.product_id;
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Recientes

3:32 p.m.

CREATE OR REPLACE T...

3:58 p.m.

Resultados de la consulta

Información del trabajo | Resultados | Detalles de la ejecución | Gra

Esta declaración creó una nueva tabla con el nombre dim_transactions_flat.

Confirmamos haciendo una vista previa de la tabla y el esquema:

dim_transactions_flat | Consulta | Cha

Esquema | Detalles | Vista previa | Estadísticas

Filtro Escribir el nombre o valor de la propiedad

Nombre del campo	Tipo	Modo
transaction_id	STRING	NULLABLE
business_id	STRING	NULLABLE
card_id	STRING	NULLABLE
timestamp	TIMESTAMP	NULLABLE
amount	FLOAT	NULLABLE
declined	INTEGER	NULLABLE
product_ids	INTEGER	REPEATED
user_id	INTEGER	NULLABLE
lat	FLOAT	NULLABLE
longitude	FLOAT	NULLABLE
product_sku	INTEGER	NULLABLE
product_name	STRING	NULLABLE
product_price	FLOAT	NULLABLE

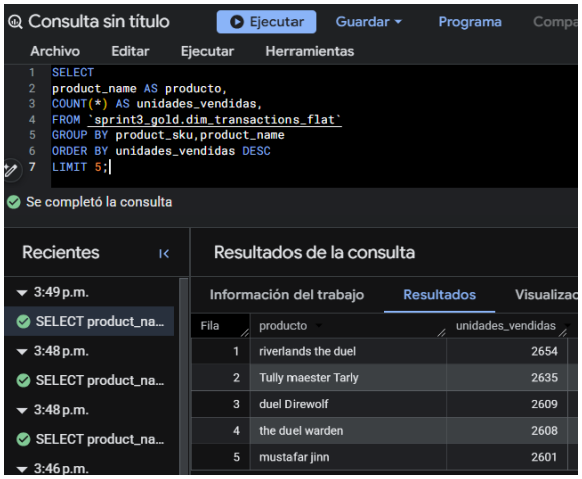
sprint3-analytics-gene / Datasets / sprint3_gold / Tables / dim_transactions_flat

dim_transactions_flat | Consulta | Chat | Abrir en | Compartir | Copiar | Instantánea | Borrar | Exportar o sincronizar

Esquema | Detalles | Vista previa | Estadísticas | Linaje | Perfil de datos | Calidad de los datos

Fila	transaction_id	business_id	card_id	timestamp	amount	declined	product_ids	user_id	lat	longitude	product_sku	product_name	product_price
1	9EDBFA19-AAD8-4F85-B72F-61...	b-2222	CcS-7897	2022-02-05 02:42:53 UTC	339.96	0	1 78	3316	46.6277244...	2.12252868...	1	Direwolf Stannis	161.11
2	66232416-7A5C-44AF-B7C1-14...	b-2222	CcU-3463	2022-09-13 06:43:03 UTC	289.29	0	43 1 60	200	54.9740133...	-3.37052822...	1	Direwolf Stannis	161.11
3	A6112549-635F-41E9-A78D-7C...	b-2222	CcS-9184	2022-06-08 05:28:24 UTC	536.03	0	44 1 65 14	4603	51.7947339...	19.2015539...	1	Direwolf Stannis	161.11
4	65781E6C-A8C3-4968-8D66-E3...	b-2222	CcU-3582	2021-07-22 15:40:27 UTC	161.11	0	1	183	54.9490497...	-3.51334804...	1	Direwolf Stannis	161.11
5	E78653F1-51CA-4AA1-838C-62...	b-2222	CcS-5493	2015-06-29 07:08:44 UTC	559.38	0	4 1 14 78	912	41.9893388...	12.6949306...	1	Direwolf Stannis	161.11
6	14141E7D-E72D-4F5A-84AB-D5...	b-2222	CcS-5544	2018-06-06 19:52:42 UTC	579.86	0	46 68 93 1	963	55.0531433...	-3.70819510...	1	Direwolf Stannis	161.11
7	0DAB4594-B24B-4783-B4EC-34...	b-2222	CcU-3666	2017-01-06 20:22:33 UTC	497.49	0	25 1 78	171	55.1275414...	-3.86120638...	1	Direwolf Stannis	161.11
8	DAA59580-1029-4D8B-AF95-08...	b-2222	CcS-6686	2018-03-29 21:42:08 UTC	394.95	0	85 1 5	2105	39.4345031...	-8.09523126...	1	Direwolf Stannis	161.11
9	887A9786-5FF6-421A-A685-CB...	b-2222	CcU-3582	2021-07-04 13:03:55 UTC	161.11	0	1	183	55.7531412...	-3.17552600...	1	Direwolf Stannis	161.11
10	0333CE96-0EBA-40C2-9E27-50...	b-2222	CcS-7116	2024-04-06 11:50:52 UTC	161.11	0	1	2535	52.3282234...	5.45831338...	1	Direwolf Stannis	161.11
11	801D719F-6BD1-43E7-B42E-F9...	b-2222	CcS-8925	2024-02-28 10:13:35 UTC	512.78	0	65	4344	59.6090779...	16.5616864...	1	Direwolf Stannis	161.11

Exercici 2: El Rànquing de Vendes (Agregació Simple):



Exercici 3: Automatització del Pipeline i Visualització:

1. **User Defined Functions (UDF):**
   Crearemos la función UDF persistente para calcular el importe del impuesto sin dejar fijados en la función ni el porcentaje de IVA ni el número de decimales, de esta forma amount representa el importe base, mientras que porcentaje_iva y decimales son parámetros configurables

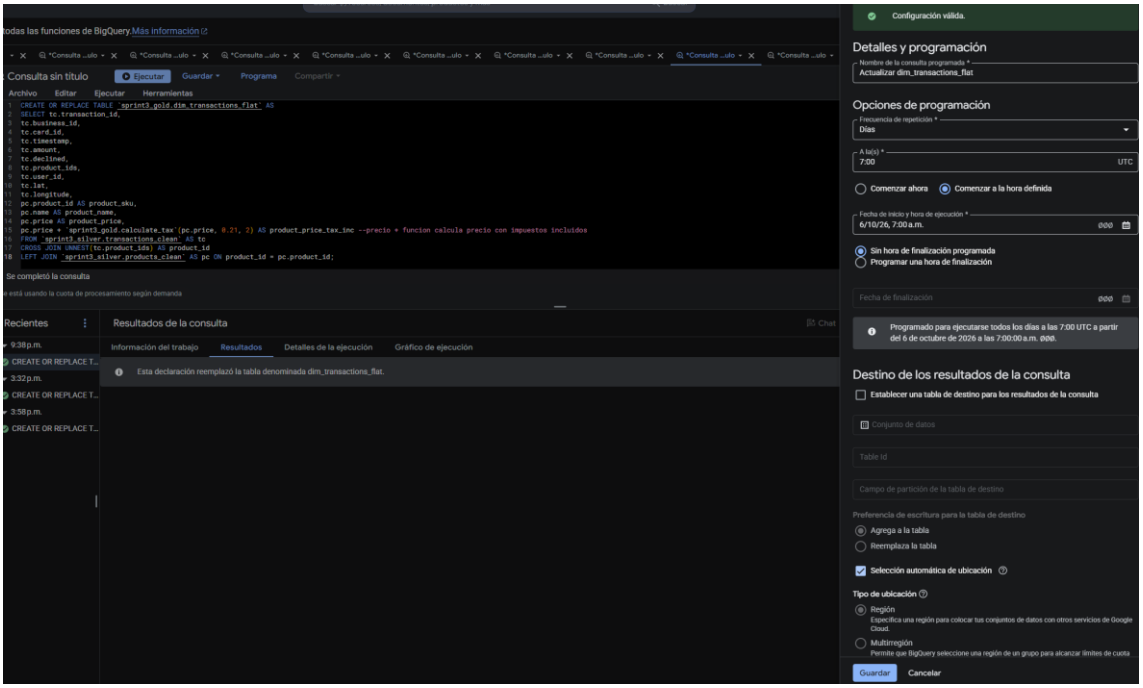
2. Integración y Orquestación

Actualizamos dim_transaction_flat añadiendo la función calculate_tax calculando el IVA:

Y una captura de la vista previa de la tabla actualizada con el IVA incluido.

Fila	transaction_id	business_id	card_id	timestamp	amount	declined	product_ids	user_id	lat	longitude	product_sku	product_name	product_price	product_price_tax_inc
1	F1DD03DB-90E0-4D02-983A-0E...	b-2290	Cc9-5776	2020-05-15 02:13:00 UTC	363.33	0	49 1 18	1195	39.95803...	-75.17436...	1	Direwolf Stannis	161.11	194.94
2	B7BBB05C-EB04-4A9C-A68D-7...	b-2590	Cc5-6118	2021-01-03 14:06:51 UTC	331.87	0	57 1	1537	39.94581...	-75.16817...	1	Direwolf Stannis	161.11	194.94
3	04A630E7-4103-4F38-95C1-410...	b-2258	Cc5-6888	2016-11-09 15:52:42 UTC	275.88	0	1	2307	46.80391...	-71.20066...	1	Direwolf Stannis	161.11	194.94

Ahora configuramos la consulta para que se ejecute cada dia a las 7:00 UTC:



3.Visualización (BI): Abrimos looker studio y cargamos nuestro modelo de datos abriendo la tabla dim_transactions_flat y configuramos un dashboard con todas las métricas del enunciado:

Enlace: [Dahsboard Datastudio]

	Producto	Unidades Vendidas	Ingresos Totales	IVA Recaudado
1.	Winterfell	10.071	1.055.850,23 €	183.241,84 €
2.	the duel	10.059	1.274.580,65 €	221.201,12 €
3.	dooku solo	10.022	695.049,04 €	120.632,65 €
4.	Karstark Dorne	7.694	852.014,74 €	147.876,72 €
5.	skywalker ewok	7.607	1.539.016,73 €	267.099,40 €
6.	Karstark warden	7.563	1.247.645,63 €	216.531,72 €
7.	Direwolf Stannis	7.429	846.455,59 €	146.905,34 €
8.	duel Direwolf	5.167	868.000,17 €	150.655,27 €
9.	duel warden	5.166	891.407,23 €	154.707,49 €
10.	Lannister	5.155	565.211,85 €	98.093,75 €

1 - 10 / 70